

Hy²: Accelerating Hybrid Mamba-Transformer Serving Through Adaptive GPU-PIM Co-Design

Anonymous Author(s)*

Abstract

Hybrid Mamba-Transformer models have emerged as a promising alternative to pure Transformers because they combine the constant-time decoding of state space models (SSMs) with the in-context recall of attention, promising higher decode throughput, smaller KV-cache footprint, and lower serving cost. However, they create a new serving challenge: different operators within the same model have different hardware affinities, the model maintains two structurally different state types (growing KV caches and fixed-size SSM states), and those affinities shift at runtime under continuous batching. GPU-only execution underutilizes GPU compute on bandwidth-bound SSM scans and attention general matrix-vector multiplications (GEMVs), while compute-bound operators in the same model still demand GPU-class throughput—making GPU-PIM architectures a natural fit. However, existing GPU serving engines assume single-device execution, and prior GPU-PIM systems largely target more uniform Transformer workloads with fixed or limited operator partitioning. We present Hy², a co-design framework for serving hybrid models on GPU-PIM hardware. Hy² jointly addresses three coupled challenges: tailoring PIM microarchitecture to the hybrid operator mix, managing heterogeneous KV-cache and SSM states across GPU and PIM memory, and adapting operator placement per layer and per iteration as continuous batching shifts batch composition. Hy² integrates workload-guided PIM design-space exploration, a unified paged-slab memory manager, and an adaptive scheduler with per-request SLO enforcement. Across five hybrid models, Hy² achieves 1.51× average throughput over prior GPU-PIM baselines and 2.47× over GPU-only baselines, while reducing tail latency to 0.61× that of prior GPU-PIM baselines.

CCS Concepts

• **Computer systems organization** → **Parallel architectures; Heterogeneous (hybrid) systems**; • **Computing methodologies** → **Neural networks**.

Keywords

Processing-in-Memory (PIM), Hardware Acceleration, Hybrid Models, LLM Serving, Mamba-Transformer Models, Dynamic Scheduling, Microarchitecture Design

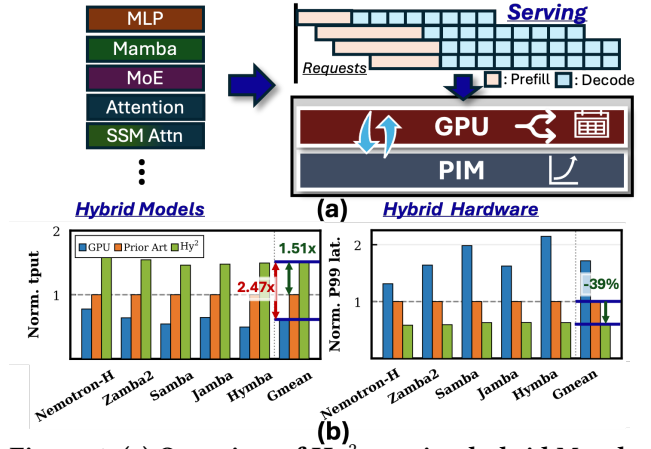


Figure 1: (a) Overview of Hy²: serving hybrid Mamba-Transformer models on GPU-PIM heterogeneous hardware. (b) Hy² throughput speedup and P99 latency reduction over GPU-only and prior art.

1 Introduction

Hybrid Mamba-Transformer models [3, 5, 8, 9, 11–13, 17, 18, 28, 42, 44, 55] have emerged as a promising alternative to pure Transformers for LLM deployment. By interleaving a majority of state space model (SSM) layers with a small number of self-attention layers, they combine the constant-time decoding of SSMs with the in-context recall strength of attention. Recent hybrid models show that this design can match or exceed Transformer quality while improving decode throughput, reducing KV-cache footprint, promising to enable long context lengths and lower serving cost across workloads ranging from reasoning [3, 5] to agentic AI [2, 4, 18]. These advantages, however, have not yet fully translated into efficient serving. During serving, especially decode, hybrid models execute a heterogeneous mix of operators: linear projections, SSM state updates, and attention computation [46]. These operators span very different arithmetic intensities and scale differently with batch size. As batching amortizes weight access, linear projections transition from memory-bound to compute-bound. In contrast, SSM scans and attention GEMVs remain memory-bound because each request accesses independent recurrent state and KV-cache data, leaving little cross-request reuse. As a result, different operators within the same model prefer hardware with very different compute-to-bandwidth ratios at runtime.

Existing systems address only part of the aforementioned problem. GPU-only serving systems [1, 27, 50, 52, 53] execute

all layer types on the same device, leaving memory-bound SSM scans and attention general matrix-vector multiplications (GEMVs) underutilized while compute resources sit idle during these operations. At the same time, hybrid models retain substantial compute-bound work — linear projections at moderate-to-large batch sizes and all prefill operations — that demands GPU-class throughput. This two-sided requirement makes GPU-PIM heterogeneous architectures a natural fit: Processing-in-Memory (PIM) hardware places lightweight compute near DRAM banks, providing bank-local bandwidth that largely exceeds the external HBM interface [24, 30] and directly targeting the bandwidth-bound operators that dominate decode time, while the GPU handles the compute-bound operators on its high-throughput cores. Yet existing GPU-PIM systems are designed for uniform Transformer workloads. Prior designs [14–16, 29, 39, 49, 51] offload a single operator type (typically attention GEMV) under a fixed or minimally adaptive partitioning, assuming homogeneous layers and stable batch sizes. Pimba [23] extends PIM acceleration to SSM and hybrid models but focuses on single-model acceleration under static batching without addressing serving-level concerns. Serving hybrid Mamba-Transformer models on GPU-PIM hardware raises three coupled co-design challenges that none of these systems address (Fig. 9):

Challenge 1 (C1): PIM microarchitecture design. Hybrid models interleave SSM and attention layers at architecture-specific ratios, and the operators offloaded to PIM span a wide range of arithmetic intensities. The PIM configuration that best serves this diverse operator mix—MAC count, buffer organization, precision—depends on both the target model architecture and the batch-size distribution encountered during serving. Existing Transformer-oriented PIM designs [15, 16, 39] target attention GEMV with a fixed microarchitecture, and Pimba [23] extends PIM to SSM operators but does not explore how PIM parameters should be jointly tuned for hybrid models. **Challenge 2 (C2): Heterogeneous state management.** Hybrid models maintain two structurally different per-request state types (Tab. 1): KV caches that grow with sequence length and are read-only during decode, and SSM states that are fixed-size but updated in-place at every token with sequential dependencies. The two state types require different allocation semantics (paged vs. slab) and different coherence handling, yet both must be placed and potentially migrated across GPU HBM and PIM memory for many concurrent requests without pipeline stalls or redundant transfers. **Challenge 3 (C3): Runtime-adaptive scheduling.** Under continuous batching with chunked prefill, the batch size fluctuates every iteration, shifting each operator’s arithmetic intensity and thus its preferred device. SSM and attention remain memory-bound regardless of batch size, but MLP’s preferred device changes as batching amortizes weight-access cost. No static

operator-to-device mapping is optimal across the range of batch sizes encountered at runtime. The scheduler must re-evaluate placement per layer and per phase while enforcing per-request latency targets.

We present **Hy²** (Hybrid model on Hybrid hardware), a co-design framework for serving hybrid Mamba-Transformer models on GPU-PIM heterogeneous hardware, as illustrated in Fig. 1-(a). Hy² makes the following contributions:

- **Hybrid model characterization.** We profile all operator types across prefill and decode phases on GPU and PIM, producing cycle-level dataflow breakdowns that quantify arithmetic intensity, bandwidth demand, and compute utilization under varying batch sizes and sequence lengths. These profiles drive both hardware design space exploration and scheduling policy design (Sec. 3) and can also improve understanding of hybrid model workloads.
- **Workload-guided PIM microarchitecture exploration.** We explore the PIM design space, sweeping MAC count, buffer organization, and precision support under DRAM process constraints, guided by the operator characterization, to identify Pareto-optimal PIM configurations for hybrid model serving (Sec. 4).
- **Unified state management for heterogeneous memory.** We design a memory management subsystem that unifies KV cache pages and SSM state blocks across GPU HBM and PIM memory, supporting efficient allocation, placement, and in-place update of both state types for concurrent requests without pipeline stalls (Sec. 5).
- **Adaptive GPU-PIM serving scheduler.** We develop a runtime scheduler that maps operators to GPU or PIM based on their profiled characteristics and the current serving load, adapting per layer and per phase as batch size and sequence length change under continuous batching, while enforcing per-request latency targets (Sec. 6).
- **Evaluation across hybrid architectures.** We evaluate Hy² on five representative hybrid models, comparing against both GPU-only and GPU-PIM baselines. Hy² achieves **1.51×** throughput over prior art and **2.47×** over GPU-only, while reducing tail latency to **0.61×** of prior art (Sec. 7).

2 Background

2.1 Hybrid Mamba-Transformer Models

Hybrid Mamba-Transformer models [3, 5, 8, 9, 11–13, 17, 18, 28, 42, 44, 55] interleave state space model (SSM) layers with self-attention layers, combining the constant-time decoding of SSMs with the in-context recall strength of attention. Each forward pass involves three or four block types (Mamba, attention, MLP, and optionally MoE) whose internal operator structures differ between prefill and decode phases (Fig. 2).

Attention blocks (Fig. 2-(a,b)) project inputs into queries, keys, and values, compute scaled dot-product attention over

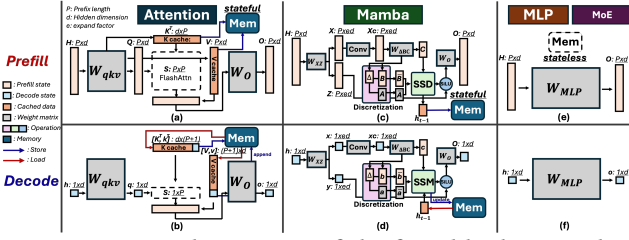


Figure 2: Internal structure of the four block types during prefill (top) and decode (bottom). Attention blocks materialize KV tensors into the cache during prefill and read the full cache during decode. Mamba blocks initialize a fixed-size recurrent state via the SSD kernel during prefill and update it in-place via the SSM scan during decode. MLP and MoE blocks contain only weight-matrix multiplications.

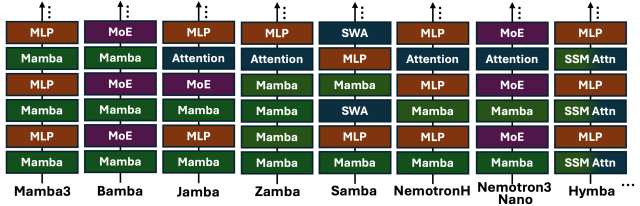


Figure 3: Repeating layer patterns of representative hybrid models. Models span a wide range of SSM-to-attention ratios and feedforward types (dense MLP vs. MoE), with Hymba (rightmost) using head-level SSM-attention fusion.

a KV cache, and apply an output projection. During prefill, the attention computation follows the FlashAttention dataflow [7], and full-sequence key and value tensors are materialized into the cache; during decode, one KV pair is appended per token and the entire cache is read for the attention computation. **Mamba blocks** (Fig. 2-(c,d)) apply input projections, a short depthwise convolution, and a core state space operation gated by SiLU and followed by an output projection. During prefill, the state space operation follows the structured state space duality (SSD) dataflow [8], processing the full input in parallel to initialize a fixed-size recurrent state; during decode, the recurrent SSM scan updates this state in-place at every token with a sequential dependency. **MLP and MoE blocks** (Fig. 2-(e,f)) apply gate, up, and down projections with a nonlinear activation; MoE blocks replace the dense MLP with a router that dispatches each token to a subset of expert MLPs (e.g., top-2 of 16 in Jamba [34]). Attention and Mamba blocks are *stateful*, carrying per-request memory across tokens with distinct properties (Tab. 1): KV caches grow linearly with sequence length and are read-only after creation, whereas SSM states are fixed-size per layer but are updated in-place with a strict sequential dependency.

These blocks are composed into full architectures by interleaving them in model-specific patterns (Fig. 3, Tab. 2). The primary design axis is the *SSM-to-attention ratio*, which

Table 1: Properties of attention and Mamba blocks.

Property	KV Cache	SSM State
Size	Grows with seq. length	Fixed per layer
Access pattern	Append + read-only	In-place update
Dependencies	None	Sequential ($t \rightarrow t+1$)
Allocation	Paged (variable size)	Slab (fixed size)

varies by over 10 $\times$ across models: from 1:1 in Samba [42] and Hymba [9] to roughly 10:1 in Bamba [17] and Granite 4.0 [18]. A secondary axis is the feedforward type: dense MLP in Nemotron-H [3], Bamba, and Samba versus MoE in Jamba [34], Granite 4.0, and HY-TurboS [44], which affects per-layer compute cost and active parameter count.

2.2 Processing-in-Memory Architectures

Processing-in-Memory (PIM) architectures integrate lightweight compute units within DRAM banks, enabling data to be processed where it resides rather than moving it across the external memory bus. Fig. 4 illustrates the architecture of an HBM-PIM system. An HBM stack consists of multiple DRAM dies connected via through-silicon vias (TSVs) to a logic die at the base, which interfaces with the host GPU. Each DRAM die is divided into pseudo-channels, each containing 16 banks that share an external bus to the logic die. Within each bank, data is read from the DRAM cell array into a row buffer, which then feeds a bank-local PIM compute unit containing multiply-accumulate (MAC) operators and a small register file for input vector storage.

The internal path from row buffer to PIM unit offers significantly higher bandwidth than the per-bank share of the external HBM bus, as all banks can operate on local data in parallel [24, 30]. Two commercial PIM chips exploit this for deep learning: Samsung’s HBM-PIM [30] and SK Hynix’s GDDR6-AiM [24], both placing FP16 MAC units within each bank at 20–27% die area overhead [6, 24, 30]. PIM compute units are fabricated in the DRAM process (e.g., 20 nm), which constrains operating frequency and area budget. GPU-PIM heterogeneous architectures leverage this complementarity by retaining compute-bound operators on the GPU while offloading memory-bound operators to PIM [15, 16, 23, 29, 39, 51].

Table 2: Representative hybrid models.

Model	Params (Active)	SSM:Attn	Attn Type	Interleaving Pattern
<i>Dense hybrids</i>				
Zamba [12]	7B	~5:1	MHA	[Mamba-1, MLP] $\times 5$, [Attn, MLP] (shared attn)
Zamba-2 [11]	1.2B–7.4B	~5:1	MHA	[Mamba-2, MLP] $\times 5$, [Attn, MLP] (shared attn)
Hymba [9]	1.5B	1:1	GQA+SWA	[Mamba-1 Attn, MLP] $\times 1$
Samba [42]	0.4B–3.8B	1:1	SWA	[Mamba-1, MLP, SWA, MLP] $\times 1$
Bamba [17]	9.8B	~10:1	GQA	[Mamba-2, MLP] $\times 10$, [Attn, MLP]
Nemotron-H [3]	8B/56B	6:1	GQA	[Mamba-2, MLP] $\times 5$, [Mamba-2, Attn, MLP]
<i>MoE hybrids</i>				
Jamba [34]	52B (12B)	7:1	GQA	[Mamba-1, MLP/MoE] $\times 7$, [Attn, MLP/MoE]
Granite 4.0 [18]	7B–32B (1B–9B)	9:1	GQA	[Mamba-2, MoE] $\times 9$, [Attn, MoE]
Nemotron 3 [5]	32B–121B (3B–13B)	~4:1	GQA	[Mamba-2, LatentMoE] $\times 4$, [Attn, LatentMoE]
HY-TurboS [44]	560B (56B)	~8:1	GQA	[Mamba-2, MoE] $\times 8$, [Attn, MoE]

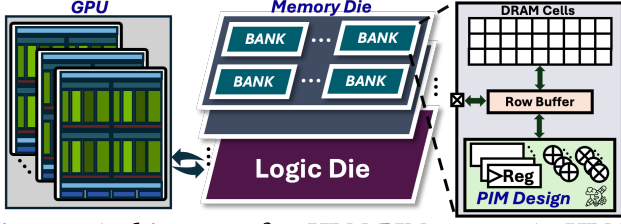


Figure 4: Architecture of an HBM-PIM system. An HBM stack comprises a logic die and multiple DRAM dies connected via TSVs. Each die contains multiple banks per pseudo-channel. The zoomed view shows a single bank: data flows from the cell array through the row buffer to a bank-local PIM compute unit.

2.3 LLM Serving

LLM inference consists of two phases (Fig. 5). During **prefill**, the model processes the full input prompt in a single forward pass, producing per-layer state tensors that are cached for subsequent generation. During **decode**, the model generates output tokens one at a time in an autoregressive loop: each iteration executes GEMVs that stream model weights and cached state from memory, making decode bandwidth-bound, whereas prefill is compute-bound due to large GEMMs over the full sequence length.

State-of-the-art serving engines are designed for GPU-only execution [1, 27, 50, 52, 53]. Early systems use **static batching**, which groups requests into a fixed batch and stalls all requests until the longest one completes. **Continuous batching** [50] removes this constraint by allowing requests to be inserted and retired at iteration granularity, but each prefill still runs as a monolithic operation: when a new request with a long prompt arrives, its prefill occupies the entire iteration, stalling all co-resident decode requests (Fig. 5-(a)). **Chunked prefill** [1] eliminates this interference by partitioning each prefill into fixed-size chunks and co-scheduling them with decode tokens in the same iteration (Fig. 5-(b)). As a result, the batch composition, the mix of prefill chunks and decode tokens—varies across iterations, shifting the effective batch size and the arithmetic intensity of each operator.

Serving engines enforce per-request **service-level objectives (SLOs)**, typically specified as upper bounds on **time-to-first-token (TTFT)**, the latency from request arrival to the first generated token, and **time-between-tokens (TBT)**, the inter-token latency during decode. Maintaining these constraints under varying load while maximizing throughput is a key scheduling problem that becomes harder when operators can be dispatched to different devices.

Memory capacity management is essential for sustaining high request concurrency. PagedAttention [27] partitions KV caches into fixed-size pages allocated on demand, reducing internal fragmentation and enabling prefix sharing

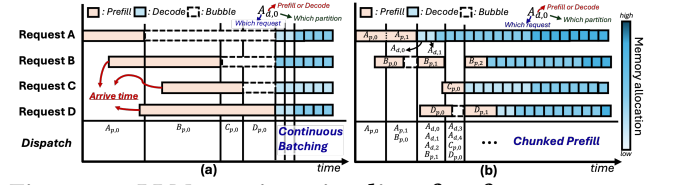


Figure 5: LLM serving timeline for four requests (A–D) arriving at staggered times. (a) Continuous batching [50]—each prefill runs as a monolithic iteration, stalling concurrent decode requests (white gaps). (b) Chunked prefill [1]—prefills are partitioned into fixed-size chunks (e.g., $B_{p,0}$, $B_{p,1}$) and co-scheduled with decode tokens in the same iteration, eliminating decode stalls. The batch composition varies per iteration, shifting the arithmetic intensity of each operator type.

across requests. As Fig. 5 illustrates, the number of active requests and their cached state grow and shrink as requests are admitted and retired, requiring the memory manager and scheduler to coordinate under continuously changing resource pressure.

3 Workload Profiling and Motivation

Efficiently serving hybrid Mamba-Transformer models on GPU-PIM hardware requires understanding **how each operator type performs on each device across the batch sizes** encountered during serving. Before the characterization, we examine where existing systems fall short. Tab. 3 classifies prior inference systems along two axes: workload heterogeneity and hardware heterogeneity. GPU-only engines [1, 27, 52, 53] handle continuous batching but execute all layers on the same device. GPU+PIM accelerators [15, 16, 29, 39, 51] offload memory-bound operators to PIM but target

Table 3: Classification of LLM inference systems. **Pure** denotes systems serving models with a single layer type (e.g., Transformer-only or SSM-only); **Hybrid** denotes systems serving models that interleave SSM and attention layers. Workload heterogeneity increases from left to right; hardware heterogeneity increases from top to bottom. The **shaded cell** marks the design space addressed by Hy².

	Workload (operator) heterogeneity			
	Low		High	
Hardware heterogeneity	Static Batching		Cont. Batching / Chunked Prefill	
	Pure	Hybrid	Pure	Hybrid
ASIC	MARCA [32], HCSAs [19], eMamba [22]	HLX [20]	–	–
PIM/PNM	CENT [14], TransPIM [54], PIM-GPT [48], HPIM [10], PIM-LLM [21], LEAP [47]	–	–	–
GPU	Mamba-1/2/3 [8, 13, 28], SpotServe [37], SpecInfer [36]	–	Orca [50], vLLM [27], SGLang [52], DistServe [53], Splitwise [40], Mooncake [41], Helix [35], Sarathi [1]	vLLM-Mamba [45]
GPU/NPU+PIM	AttAcc [39], NeuPIMs [16], IANUS [43], PAPI [15], SpecPIM [31], FAISE [29], Pyramid [49], LoL-PIM [26], ORCHES [33]	Pimba [23]	Duplex [51], Hybe [38] [†]	Hy ² (Ours)

[†]Hybe is GPU+NPU (not PIM); grouped as heterogeneous.

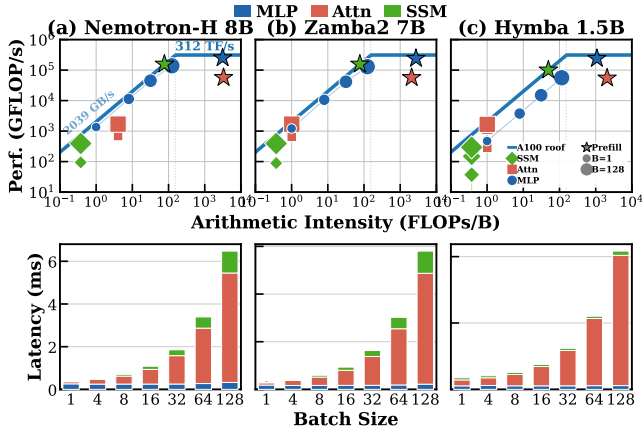


Figure 6: Measured A100 GPU characterization ($s=4096$). Top: Roofline with compute ceiling (312 TF/s) and HBM BW slope (2,039 GB/s); circles/squares/diamonds = decode at $B \in \{1, 8, 32, 128\}$, stars = prefill ($B=8$). Bottom: Per-layer decode latency by operator category.

pure Transformers; Pimba [23] extends PIM to SSM and hybrid models but lacks serving-level support. The combination of hybrid models under continuous batching on GPU-PIM hardware remains unaddressed, which is the design point Hy² targets.

The rest of this section profiles hybrid-model operators on GPU and PIM using a joint roofline analysis (Sec. 3.1), examines how continuous batching shifts operator behavior at runtime (Sec. 3.2), and derives three co-design challenges that shape the Hy² architecture (Sec. 3.3).

3.1 Operator Profiling and Roofline Analysis

We select three representative models from Tab. 2: Nemotron-H 8B, Zamba2 7B, and Hymba 1.5B. For each model we isolate three main operator categories: MLP, attention, and SSM state update. We sweep batch size $B \in \{1, 4, 8, 16, 32, 64, 128\}$ during decode and profile prefill at $B=8$, both at sequence length $s=4096$, on two platforms: an NVIDIA A100-SXM4-80 GB GPU (312 TF/s FP16, 2,039 GB/s HBM2e) and a GPU+PIM system modeled after Pimba [23] (20 TF/s effective PIM compute, 25.6 TB/s aggregate bank bandwidth).

3.1.1 GPU Profiling. Fig. 6 presents the measured A100 results. Four patterns emerge. (i) **SSM and attention stay memory-bound at all B .** SSM state updates remain at $AI = 0.38$ and attention GEMV at $AI = 4.0$ regardless of batch size, because each request accesses its own state with no cross-request data reuse. (ii) **MLP crosses the compute-bound threshold with B .** At $B=1$, MLP is memory-bound ($AI = 1.0$), but its intensity grows linearly with B due to weight reuse across requests. (iii) **Prefill is compute-bound for all operators.** MLP reaches 81% of peak; Mamba-2’s SSD

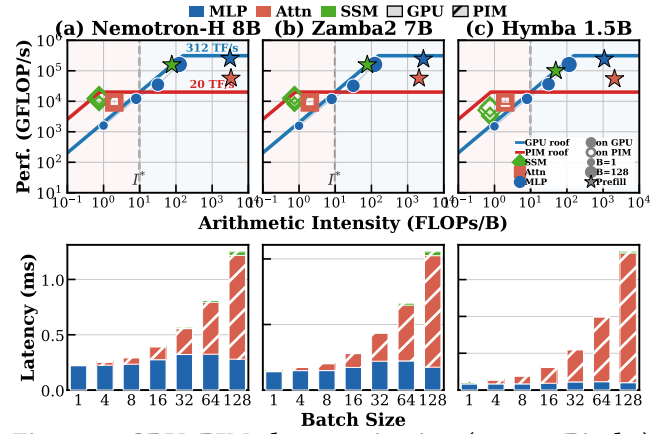


Figure 7: GPU+PIM characterization ($s=4096$, Pimba). Top: GPU-PIM roofline overlaying GPU (blue, 312 TF/s compute, 2,039 GB/s HBM) and PIM (red, 20 TF/s compute, 25.6 TB/s bank BW). Dashed line marks the scheduling threshold $I^* \approx 9.8$ FLOPs/B: operators left of I^* are PIM-preferred, right are GPU-preferred. Hollow markers = PIM decode, filled = GPU decode, stars = prefill ($B=8$, all on GPU). Bottom: Per-layer decode latency by operator category.

dataflow [8] reformulates the sequential scan as chunked matrix multiplications, reaching $AI \approx 49\text{--}77$ across models. (iv) **Latency breakdown shifts with B .** Fig. 6 (bottom) shows the per-layer decode latency. At low B , MLP dominates because all operators are memory-bound and MLP has the largest weight footprint. As B grows, SSM and attention latency scales linearly (no cross-request reuse), while MLP latency barely increases due to weight reuse. At $B=128$, SSM and attention account for 84% of full-model GPU-only decode time (Tab. 4).

3.1.2 GPU+PIM Profiling and Combined Roofline. The GPU-only roofline shows which operators are memory-bound but does not reveal where PIM is preferable. We overlay two independent rooflines, one per device, each defined by its own peak compute and local bandwidth. Fig. 7 (top) shows four ceilings: GPU compute (312 TF/s), GPU HBM bandwidth (2,039 GB/s), PIM aggregate bank bandwidth (25.6 TB/s), and

Table 4: Full-model decode-time breakdown for Nemotron-H 8B ($s=4096$). Figures 6–7 show per-layer latency; this table multiplies by layer count (24 MLP, 24 SSM, 4 Attn).

B	GPU-only (ms)				GPU+PIM (ms)				Speedup
	MLP	SSM	Attn	Total	MLP ^G	SSM ^P	Attn ^P	Total	
1	6.2	0.8	0.4	7.4	6.2	0.01	0.03	6.2	1.2×
8	6.0	2.0	1.5	9.5	6.0	0.04	0.24	6.3	1.5×
32	6.0	6.6	5.3	17.9	6.0	0.20	0.94	7.1	2.5×
64	6.6	12.6	10.4	29.6	6.6	0.40	1.88	8.9	3.3×
128	7.8	24.6	20.5	52.9	7.8	0.78	3.76	12.3	4.3×

^G GPU ^P PIM (Pimba cycle-level simulation)

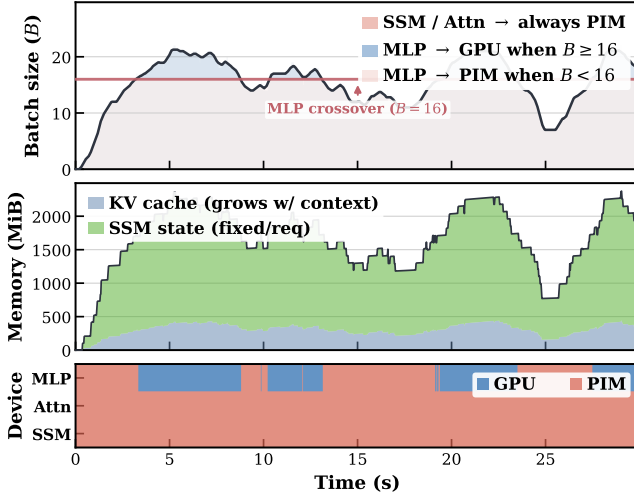


Figure 8: Nemotron-H 8B under continuous batching (Poisson arrivals, 2.8 req/s, chunked prefill $C=256$, prompt length 512, decode length $\mu=250$, $\sigma=80$). Top: Running batch size B ; red line marks the MLP GPU/PIM crossover ($B=16$). Middle: Aggregate per-request state memory (SSM state vs. KV cache). Bottom: Per-operator device assignment heatmap.

PIM compute (20 TF/s). The two rooflines cross at the *scheduling threshold* $I^* \approx 9.8$ FLOPs/B (dashed vertical line): operators with arithmetic intensity below I^* are PIM-preferred; those above I^* are GPU-preferred. Three findings emerge. (i) **SSM and attention are PIM-preferred.** SSM ($AI \leq 0.75$) and attention ($AI \leq 4$) sit far below I^* at every batch size. Compared to the measured A100 GPU (Sec. 3.1.1), SSM on PIM is 94 $\times$ faster at $B=1$ and 34 $\times$ at $B=32$; attention GEMV on PIM is 14.5 $\times$ faster at $B=1$ and 5.7 $\times$ at $B=32$. (ii) **MLP’s preferred device depends on B .** MLP’s arithmetic intensity grows linearly with batch size, crossing I^* around $B=10$ –16 depending on the model and operator mix: below this threshold MLP is PIM-preferred; above it, GPU becomes the preferred device. (iii) **Full-model speedup grows with B .** Tab. 4 shows the full-model decode-time breakdown for Nemotron-H. On GPU-only, SSM and attention account for 66% of decode time at $B=32$ and 85% at $B=128$. Offloading them to PIM yields full-model speedups of 2.5 $\times$ and 4.3 $\times$, respectively. However, after PIM offloading MLP becomes the bottleneck ($\geq 63\%$ of hybrid time at every B), making MLP scheduling the key remaining challenge.

3.2 Continuous Batching Simulation

The profiling in Sec. 3.1.1 and Sec. 3.1.2 characterizes operators at fixed batch sizes. However, under continuous batching with chunked prefill [1, 50], the batch composition shifts every iteration. To quantify this effect, we simulate Nemotron-H 8B serving under Poisson arrivals (mean 2.8 req/s) with

chunked prefill ($C=256$, prompt length 512) and variable decode lengths ($\mu=250$, $\sigma=80$). Fig. 8 presents the results. This leads to four key observations.

(i) **Batch size fluctuates continuously.** B oscillates between 4 and 22 (Fig. 8, top), crossing the MLP crossover ($B \approx 16$) in 51% of iterations. SSM and attention remain PIM-preferred at every B ; only MLP’s placement is sensitive to batch size. (ii) **No static mapping is optimal.** Full-model speedup ranges from 1.2–1.5 $\times$ at low B to 4.0 $\times$ at high B (Tab. 4). MLP alternates between GPU and PIM as B fluctuates (Fig. 8, bottom); a fixed assignment either wastes PIM bandwidth or leaves GPU compute idle. (iii) **SSM state dominates per-request memory in SSM-heavy models.** For Nemotron-H, SSM state is fixed at 48 MiB/req (24 layers $\times$ 2 MiB) and exceeds KV cache until $\sim 3,100$ tokens; at typical lengths (~ 750 tokens) it is $\sim 4.0\times$ larger (Fig. 8, middle). The balance reverses for architectures with head-level SSM–attention fusion (e.g. Hymba), where KV cache dominates instead (Tab. 5).

3.3 Challenges and System Overview

The profiling (Sec. 3.1) and simulation (Sec. 3.2) expose three co-design challenges. Fig. 9 provides an overview of each challenge and the corresponding technique in Hy².

C1: PIM microarchitecture must be tailored to the dynamic hybrid-model workload (Sec. 3.1). Operators offloaded to PIM range from SSM scans ($AI = 0.38$) to low-batch MLP ($AI \approx 1$ –10), and the optimal MAC count and buffer organization vary with the model’s SSM-to-attention ratio and the batch-size distribution encountered during serving. $\Rightarrow$ *Workload-aware PIM DSE* (Sec. 4).

C2: Memory allocation and weight placement must account for heterogeneous state (Sec. 3.1, finding iii). GPU and PIM share HBM but access it through different paths: GPU via the channel interface and PIM via bank-local datapaths. Model weights must be distributed so that each operator’s data is local to its preferred compute unit. Beyond weights, each request carries KV caches (context-dependent) and SSM states (fixed-size), which compete for capacity and require different allocation and coherence semantics. $\Rightarrow$ *Unified memory manager* (Sec. 5).

C3: Operator placement must adapt at runtime (findings i, ii). Because B fluctuates every iteration under continuous batching, the optimal operator-to-device mapping shifts accordingly. $\Rightarrow$ *Adaptive GPU-PIM scheduler* (Sec. 6).

4 PIM Microarchitecture Design Space

The characterization in Sec. 3 shows that hybrid models contain both bandwidth-bound operators (SSM, attention at low B) suited to PIM and compute-bound operators (MLP at moderate B) suited to GPU. Fig. 10 illustrates the PIM bank

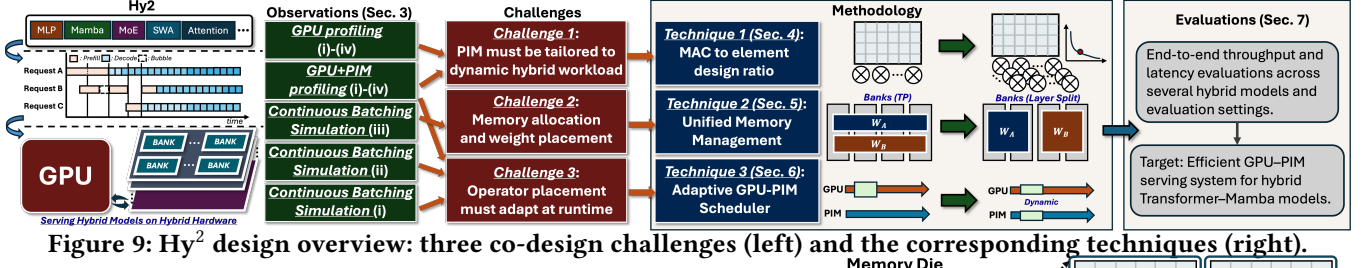


Figure 9: Hy² design overview: three co-design challenges (left) and the corresponding techniques (right).

microarchitecture: each DRAM bank houses a small array of multiply-accumulate (MAC) units that process data locally, avoiding the external memory bus. A natural partitioning assigns MLP to the GPU and SSM/attention to PIM, with each operator’s weights stored on its respective device (formalized as *layer-split* placement in Sec. 5.1). In HBM-PIM, bank groups designated for PIM operate on bank-local datapaths while the remaining bank groups serve GPU requests through the standard HBM channel interface [16, 29, 30]. Because these two datapaths are independent, the GPU and PIM pipelines execute **concurrently**:

$$t_{\text{iter}} = \max \left(\underbrace{T_{\text{MLP}}^{\text{GPU}}}_{\text{GPU}}, \underbrace{T_{\text{SSM}}^{\text{PIM}} + T_{\text{Attn}}^{\text{PIM}}}_{\text{PIM}} \right). \quad (1)$$

This overlap transforms the PIM provisioning question from “minimize the total latency” to “balance the two pipelines.” The key design variable is how much compute to provision on the PIM side: too little and PIM becomes the bottleneck; too much and the extra silicon area is wasted because GPU MLP already dominates the critical path. Crucially, this balance is *not static*. Under continuous-batching serving, the request batch size B fluctuates every iteration as new requests arrive and completed ones retire. Because GPU and PIM latencies scale differently with B (Sec. 3), the pipeline balance shifts at runtime, a dimension that prior single-inference PIM studies [16, 23, 39] do not capture.

4.1 Design Space and Methodology

Key design parameter: MAC-to-element ratio R . Each DRAM bank houses a small array of multiply-accumulate (MAC) units (Fig. 10), and the number of MACs determines what operations PIM can perform efficiently. A standard HBM bank read produces 256 bits (32 bytes), which for INT8 data yields $n=32$ elements, one per MAC in the baseline design [30]. We define the **MAC-to-element ratio R** as the number of MACs per data element from a single bank read. Increasing R replicates the MAC array and adds activation registers to cache R input vectors, so each bank read drives R parallel dot products, enlarging only the compute logic while the DRAM array remains unchanged (Fig. 10).

- $R=1$: **GEMV baseline**. Each bank read activates exactly one MAC per element. PIM can efficiently execute GEMV

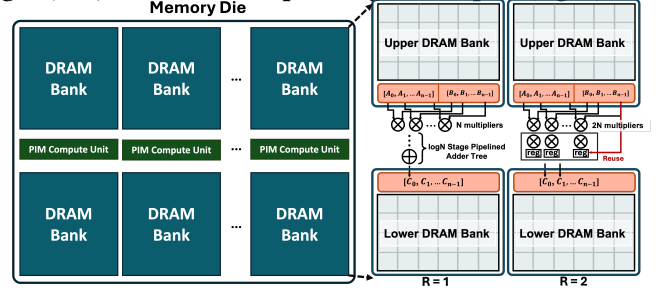


Figure 10: PIM bank microarchitecture and the effect of MAC ratio R . Left: DRAM banks with PIM compute units. Center ($R=1$): A bank read produces n weight elements $[A]$; n MACs multiply them against one activation vector $[B]$, producing one output row $[C]$ (GEMV). Right ($R=2$): The same weight read is *reused*: $2n$ MACs multiply $[A]$ against two cached activation vectors via additional multipliers and registers, producing two output rows per bank cycle. Increasing R enlarges the compute logic; the DRAM array is unchanged.

operations (SSM state updates, attention QK^T and PV) but not matrix-matrix multiplications.

- $R=2$: **partial GEMM**. Two activation vectors are cached; each weight read drives two parallel dot products, enabling PIM to handle small-batch MLP.
- $R=4$: **GEMM-capable PIM**. Four parallel dot products per bank read enable significant MLP throughput on PIM.

We sweep $R \in \{1, 2, 4\}$ across 1,280 banks. PIM bandwidth is 25.6 TB/s; compute scales as $32R \times 1280 \times f_{\text{HBM}}$ ops/s. With one PIM compute unit per two banks, the per-bank area overhead is 19% ($R=1$), 38% ($R=2$), and 80% ($R=4$) of the DRAM bank area [30], driven entirely by MAC-array and register replication.

Overlap-aware methodology. For each (MAC count, B), we compute PIM latency using a roofline model calibrated to Pimba cycle-level simulation. MLP runs on GPU; SSM and attention run on PIM. Iteration latency follows Eq. 1.

4.2 Pipeline Balance Analysis

Fig. 11 sweeps R from sub-baseline ($R < 1$, fewer MACs than the standard HBM-PIM bank) through $R \in \{1, 2, 4\}$ at $B=1, 16$, and 128. **Finding 1:** At $R < 1$ and high B , PIM becomes the bottleneck because insufficient MACs cannot keep up with the linearly growing SSM+attention workload. **Finding 2:**

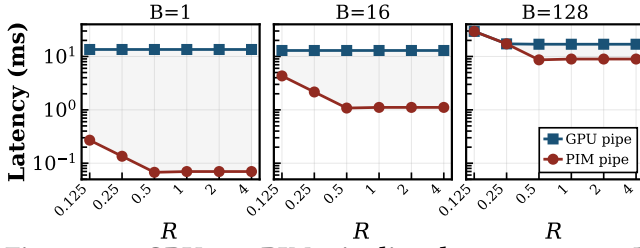


Figure 11: GPU vs. PIM pipeline latency across R (Nemotron-H 8B). $R < 1$ denotes sub-baseline MACs; $R \geq 1$ are the design points in this work. PIM bottlenecks at low R and high B ; at $R \geq 1$ GPU MLP dominates.

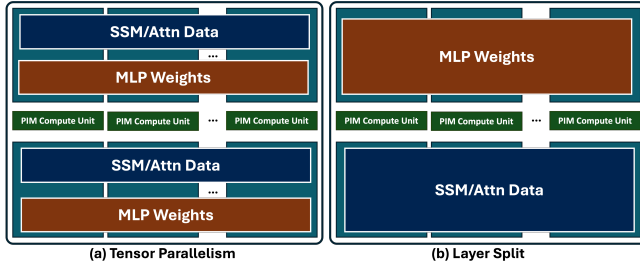


Figure 12: Weight placement strategies. (a) Tensor parallelism: every bank stores both MLP and SSM/attention weights; AllReduce synchronization is required after each layer. (b) Layer-split: MLP weights reside in normal-mode banks (accessed by GPU via the HBM channel), SSM/attention weights reside in PIM-mode banks (processed locally by PIM compute units). No weight replication, no AllReduce; $t_{\text{iter}} = \max(T_{\text{GPU}}, T_{\text{PIM}})$.

At $R \geq 1$, PIM SSM+attention (4.54 ms at $B=128$) is well below GPU MLP (7.82 ms); PIM latency is fully hidden by overlap. GPU MLP dominates the critical path across all batch sizes, confirming that even the GEMV-baseline ($R=1$) is sufficient for SSM and attention offloading. **Finding 3:** At $R > 1$, surplus MACs enable MLP on PIM, creating a two-dimensional scheduling problem over both SSM/attention and MLP fractions. This batch-dependent imbalance motivates an adaptive scheduler that rebalances the two pipelines every iteration.

The pipeline balance depends on R , but under continuous-batching serving, B fluctuates every iteration, so the optimal trade-off between area and scheduling flexibility cannot be determined from a static analysis alone.

5 Unified Memory Management

GPU and PIM share HBM via different datapaths: GPU through channel controllers, PIM through bank-local paths. **Concurrent execution requires weights and state to reside on the correct side of the GPU/PIM boundary.** This section covers weight placement (Sec. 5.1) and memory pressure (Sec. 5.2).

5.1 Weight Placement: TP vs. Layer-Split

Fig. 12 compares two weight placement strategies. **Tensor parallelism (TP)** replicates every layer’s weights across both GPU-accessible and PIM-mode banks, so both devices access the *same* bank groups. Each forward pass therefore requires AllReduce synchronization, and GPU and PIM cannot execute concurrently. **Layer-split** assigns entire layers to one device: MLP weights in normal-mode banks, SSM/attention weights in PIM-mode banks. Because normal-mode banks connect to the GPU through the HBM channel interface while PIM-mode banks use bank-local datapaths, the two pipelines are physically independent and execute concurrently (Eq. 1). No weight replication is needed, and the freed capacity is available for request state.

5.2 Memory Pressure

Tab. 5 shows per-request memory at $s=4096$. For Nemotron-H, SSM state (48 MiB) exceeds KV cache below $\sim 3,100$ tokens; at typical serving lengths both types contribute significantly. Concurency drops $3.1\times$ from $s=512$ (1,154 reqs) to $s=8192$ (367 reqs). Under layer-split, SSM+attention weights (3.76 GB) reside in PIM-mode banks; MLP weights (8.46 GB) in normal-mode banks.

6 Adaptive GPU-PIM Scheduler

Under continuous batching the batch size B fluctuates every iteration. The scheduler recomputes closed-form GPU-PIM split fractions per iteration, routing each operator type to its preferred device based on the current B .

6.1 Core Algorithm: Overlap + Adaptive Split

At $R=1$ (GEMV): MLP stays on GPU; only SSM/attention placement is adjustable. Let $G=T_{\text{MLP}}^{\text{GPU}}$, $g=T_{\text{SSM+Attn}}^{\text{GPU}}$, $p=T_{\text{SSM+Attn}}^{\text{PIM}}$. The scheduler assigns a fraction x of SSM/attention heads to PIM and $(1-x)$ to GPU, yielding iteration latency $\max(\text{GPU}, \text{PIM})$ where:

$$\underbrace{G + (1-x)g}_{\text{GPU}} \text{ vs. } \underbrace{x \cdot p}_{\text{PIM}}, \quad x^* = \frac{G+g}{g+p} \in [0, 1]. \quad (2)$$

Table 5: Per-request state memory (FP16) at $s=4096$ and KV-SSM crossover. Llama-2 (pure Transformer) and Mamba-2 (pure SSM) are included as boundary cases.

Model	KV (MiB)	SSM (MiB)	Total (MiB)	Cross. (tok)	Max (reqs)
Llama-2 7B	2048	0	2048	–	32
Mamba-2 2.8B	0	80	80	–	859
Nemotron-H 8B	64	48	112	3100	577
Hymba 1.5B	384	1.5	386	16	194

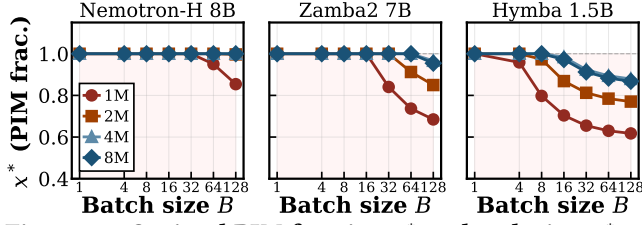


Figure 13: Optimal PIM fraction x^* vs. batch size. $x^* < 1$ (shaded) indicates head-level splitting activates.

Table 6: Prefill latency breakdown per chunk ($C=256$ tokens) and TTFT reduction from disaggregation (2 chunks). Llama-2 and Mamba-2 serve as boundary cases (pure Transformer and pure SSM).

Model	GPU MLP	GPU SA	PIM SA	Disagg	TTFT reduct.
		(ms/chunk)			
Llama-2 7B	5.92	4.44	0.67	5.92	42.9%
Hymba 1.5B	0.83	0.32	0.14	0.83	27.8%
Nemotron-H 8B	8.67	0.61	0.59	8.67	6.5%
Mamba-2 2.8B	4.65	0.09	0.84	4.65	1.8%

When $x^* \geq 1$, GPU MLP dominates and no split is needed (all SSM/attention runs on PIM). When $x^* < 1$ (PIM bottleneck), fraction x^* of heads go to PIM and $(1-x^*)$ to GPU.

At $R > 1$ (GEMM): MLP can also be split. Let $M = T_{\text{MLP}}^{\text{PIM}}$. The scheduler optimizes two fractions (x_p, x_m):

$$\underbrace{(1-x_m)G + (1-x_p)g}_{\text{GPU}} \text{ vs. } \underbrace{x_p \cdot p + x_m \cdot M}_{\text{PIM}}. \quad (3)$$

At each iteration, the scheduler evaluates the closed-form x^* (or (x_p, x_m) at $R > 1$) using calibrated latency tables. The decision takes $< 50 \mu\text{s}$ ($< 0.3\%$ of an iteration).

Fig. 13 shows x^* across batch sizes. For most hybrid models $x^* = 1$ (all SSM/attention on PIM), confirming that GPU MLP dominates (Finding 1, Sec. 4.2). Hymba is the exception: its small MLP leaves GPU underutilized, activating head-level splitting at $B \geq 16$ (up to $1.62\times$ speedup, Fig. 14).

6.2 Prefill-Decode Disaggregation

Hy² extends the overlap principle to chunked prefill: GPU handles MLP prefill (GEMM) while PIM handles SSM+attention prefill (GEMV) concurrently, giving $t_{\text{pf}} = \max(T_{\text{MLP}}^{\text{pf}}, T_{\text{SSM}}^{\text{pf}} + T_{\text{Attn}}^{\text{pf}})$.

Tab. 6 shows per-chunk cost. TTFT reduction scales with attention count: 42.9% for Llama-2 (32 attn layers), 27.8%

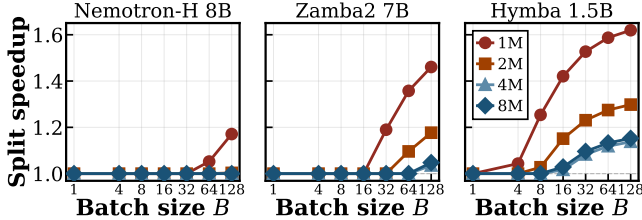


Figure 14: Speedup from adaptive split over no-split overlap.

for Hymba, but only 6.5% for Nemotron-H (MLP-dominated prefill).

6.3 GEMM-Regime 2D Split ($R > 1$)

At $R > 1$, the scheduler optimizes (x_p, x_m) per Eq. 3.

Tab. 7 shows the 2D split at $R=2$. At low B , PIM’s bandwidth advantage routes 37–83% of MLP to PIM (up to $5.73\times$ over 1D at $B=1$). As B grows, MLP becomes compute-bound and $x_m \rightarrow 0$, converging to the 1D split. The 2D split is what distinguishes $R=2$ from $R=1$ and is the primary source of Hy²’s speedup over lockstep (Sec. 7.4).

7 Evaluation

7.1 Experimental Setup

Models. We evaluate five hybrid models spanning diverse layer compositions, as summarized in Tab. 8. All models interleave SSM and attention layers with MLP, but differ in the ratio and count of each layer type.

Hardware. A100-80 GB GPU paired with PIM (25.6 TB/s aggregate bandwidth). Results are shown at $R=2$. **Simulation.** PIM latencies from Pimba Ramulator2 [23]. **Serving methodology.** The comprehensive evaluation uses closed-loop simulation (always B active requests with continuous batching and chunked prefill at $C=256$), matching prior architecture evaluations [51]. **Baselines.** GPU-only, Pimba, and Hy². Since Pimba targets single-model inference under static batching, we extend it with continuous batching and chunked prefill to provide a fair GPU-PIM baseline. All three use the same serving framework.

7.2 End-to-End Serving Performance

Fig. 15 reports throughput and P99 TBT for five hybrid models (Tab. 8), sweeping batch size $B \in \{4, 8, 16\}$ and context length $\{512, 1k, 2k\}$. All values are normalized to Pimba.

Throughput. Hy² improves throughput over Pimba by $1.51\times$ on geometric mean across all 45 configurations, and $2.47\times$ over GPU-only. The source of this gain is the 2D MLP split (Sec. 6.3): at $R=2$, Hy² overlaps PIM-side MLP with GPU compute, converting what is sequential in Pimba into a balanced pipeline. Nemotron-H achieves the highest per-model mean ($1.60\times$), followed by Zamba2 ($1.54\times$), Hymba

Table 7: Optimal (x_p, x_m) and speedup over 1D split (GEMV-only) and lockstep at $R=2$ for Nemotron-H 8B.

B	x_p	x_m	PIM MLP (ms)	Iter (ms)	vs. 1D	vs. Lock
1	1.00	0.83	0.90	0.92	$5.73\times$	$1.21\times$
4	1.00	0.63	1.90	1.99	$2.71\times$	$1.56\times$
16	1.00	0.37	3.80	4.14	$1.59\times$	$1.67\times$
32	1.00	0.24	5.20	5.88	$1.31\times$	$1.43\times$
64	1.00	0.12	5.46	6.83	$1.14\times$	$1.34\times$
128	1.00	0.04	3.68	6.42	$1.04\times$	$1.47\times$

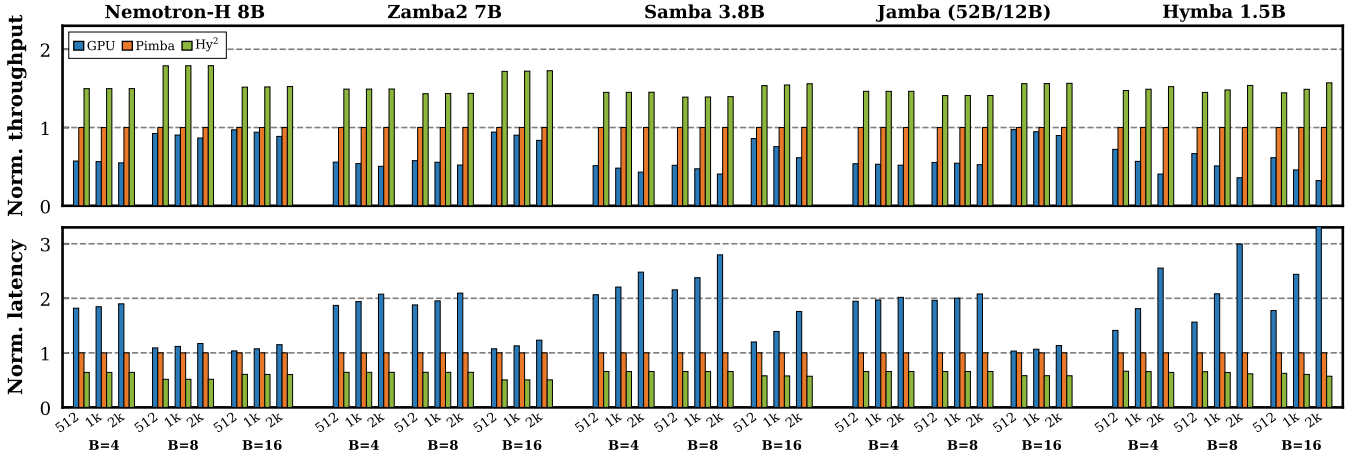


Figure 15: Normalized throughput (top) and P99 TBT (bottom) vs. Pimba across five hybrid models at $R=2$ under closed-loop serving with $B \in \{4, 8, 16\}$ and context length $\{512, 1k, 2k\}$. Hy^2 achieves $1.51\times$ higher throughput and 39% lower P99 TBT than Pimba on average.

Table 8: Evaluated hybrid models and compositions.

Model	Type	MLP	SSM	Attn	d	Size
Nemotron-H [3]	Hybrid	24	24	4	4096	8B
Zamba2 [11]	Hybrid	76	76	13	3712	7B
Samba [42]	Hybrid	24	12	12	4096	3.8B
Jamba [34]	Hybrid-MoE	48	28	4	4096	52B/12B
Hymba [9]	Hybrid	32	16	16	1536	1.5B

($1.49\times$), Jamba ($1.48\times$), and Samba ($1.46\times$). All five models consistently exceed $1.39\times$ in every configuration.

Scaling with batch size and context. The advantage grows with both batch size and context length. At $B=16$, the speedup rises to $1.57\times$ vs. $1.48\times$ at $B=4$, because larger batches increase the MLP-to-attention ratio, making the overlap more effective. Context scaling has a complementary effect: at $\text{ctx}=2k$ the mean reaches $1.53\times$ vs. $1.50\times$ at $\text{ctx}=512$, as the growing attention cost penalizes GPU-only ($2.80\times$ behind Hy^2) far more than PIM-equipped schemes.

Latency. The normalized P99 TBT (bottom row) mirrors the throughput trend. Hy^2 reduces P99 TBT to $0.61\times$ of Pimba on average, reflecting the per-iteration speedup from overlap. GPU-only latency inflates at larger context, reaching $3\times$ Pimba at Hymba $B=16$, $\text{ctx}=2k$, because the attention GEMV is bandwidth-bound on GPU but efficiently handled by PIM in both Pimba and Hy^2 .

7.3 Arrival Rate and Tail Latency

Fig. 16 isolates the effect of arrival rate and context length on Nemotron-H. As λ grows from 1 to 20 req/s, Hy^2 's per-iteration advantage compounds: at $\lambda=20$, Hy^2 reaches 15,217 tok/s vs. Pimba's 14,723 ($1.03\times$) and GPU's 12,902 ($1.18\times$). Hy^2 consistently achieves the lowest P50 TBT (3.1 ms vs. Pimba's 7.1 ms at $\lambda=10$). The P90 and P99 panels reveal queueing dynamics: at $\lambda=20$, GPU P99 crosses the 100 ms SLO (108.8 ms) while Hy^2 remains at 37.7 ms and Pimba at 55.8 ms. At $\lambda=15$,

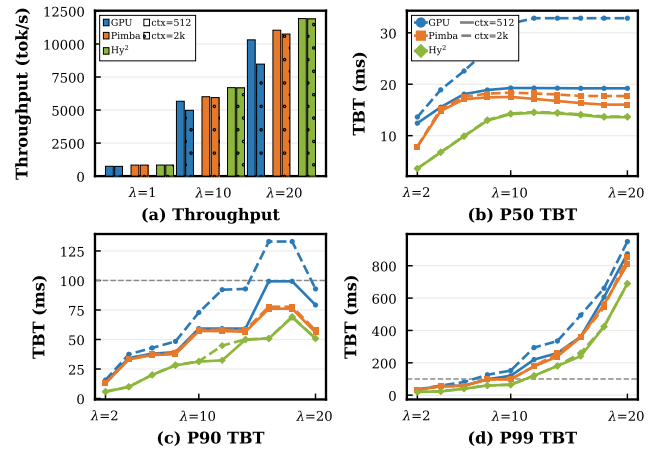


Figure 16: Nemotron-H 8B ($R=2$) under Poisson arrivals at two context lengths ($\text{ctx}=512$ solid, $\text{ctx}=2k$ dashed). (a) Throughput at $\lambda \in \{1, 10, 20\}$. (b-d) P50, P90, and P99 TBT swept from $\lambda=2$ to 20.

Hy^2 (22.6 ms) maintains a 39% margin over Pimba (37.1 ms) and 68% over GPU (71.4 ms).

7.4 Hardware MAC Sensitivity

Prior work selects PIM configurations under fixed batch sizes and single-model inference [16, 23]. Under serving, however, continuous batching causes B to fluctuate every iteration, shifting the GPU-PIM balance dynamically. This makes scheduling flexibility—not just raw throughput—the decisive factor in choosing R . Fig. 17 sweeps R at $\lambda=20$ while holding other parameters fixed.

Hy^2 absolute throughput is stable across R . From $R=1$ to $R=4$, Hy^2 throughput changes by 3.6% (14,741 vs. 15,273 tok/s) because overlap already hides PIM SSM/Attention latency behind GPU MLP.

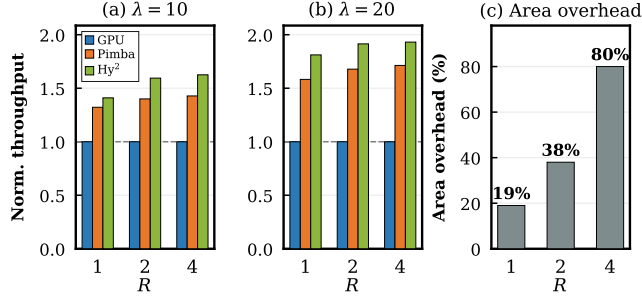


Figure 17: MAC sensitivity across $R \in \{1, 2, 4\}$ for Nemotron-H. (a, b) Normalized throughput at medium ($\lambda=10$) and high ($\lambda=20$) request rates. (c) Per-bank PIM area overhead.

The per-iteration speedup over Pimba, however, depends on R . At $R=1$, PIM can only execute GEMV (SSM, attention), so the 2D MLP split (Sec. 6.3) cannot activate. Per-iteration latency speedup is $1.26\times$ over Pimba on geometric mean across five models—overlap alone provides modest headroom when MLP cannot be shared (Fig. 17b). At $R=2$, the 2D split routes a fraction of MLP to PIM, and the speedup jumps to $1.61\times$. At $R=4$, the speedup drops to $1.54\times$ because Pimba itself improves with more compute.

Selected configuration. We select $R=2$ as the primary PIM configuration. $R=2$ is the minimum R that enables PIM MLP and the 2D scheduling split, achieving the *highest* speedup over prior art at half the area of $R=4$ (Fig. 17-c)). This confirms that serving-level R selection requires evaluating scheduling flexibility under dynamic batch sizes, not just static throughput or throughput/mm².

7.5 Scheduling Comparison

Fig. 18-(a) shows that Hy² achieves the lowest iteration latency at every B : at $B=1$ it closely tracks Pimba (PIM handles most work), while at $B \geq 8$ it pulls ahead as the 2D split distributes MLP across both devices. GPU-only latency grows to 25 ms at $B=128$ because it cannot offload any work; Pimba saturates at ~ 9 ms due to sequential execution. Fig. 18-(b) reveals the mechanism: the MLP split ratio x_m drops monotonically from 83% to 4% as B grows. At low B , PIM compute is underutilized, so the scheduler routes most MLP to PIM; at high B , PIM MLP becomes expensive and the scheduler routes nearly all MLP back to GPU. This continuous adaptation is why Hy² maintains its advantage across the full batch-size range encountered under continuous batching.

8 Related Work

LLM serving systems. State-of-the-art GPU serving engines have progressively eliminated sources of inefficiency in Transformer inference. Continuous batching [50] allows requests to enter and exit at iteration granularity, and PageAttention [27] reduces KV cache fragmentation through

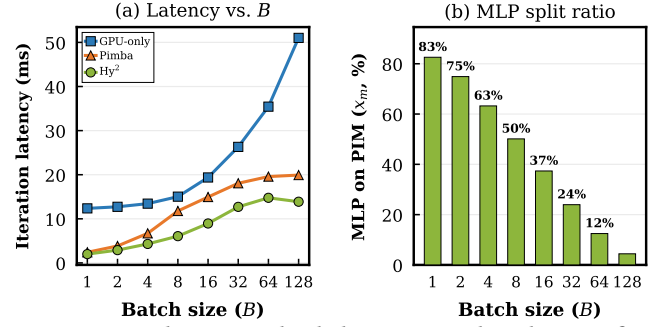


Figure 18: Adaptive scheduling across batch sizes for Nemotron-H at $R=2$. (a) Per-iteration latency. (b) MLP split ratio x_m .

on-demand page allocation. A remaining bottleneck is prefill-decode interference: chunked prefill [1] addresses this by partitioning prefills into fixed-size chunks co-scheduled with decode tokens, while prefill-decode disaggregation [40, 53] isolates the two phases on separate GPU pools. Complementary optimizations target structured generation [52] and heterogeneous GPU clusters [35, 41]. These systems are designed for GPU-only execution and do not exploit the cross-layer hardware heterogeneity on GPU-PIM that Hy² targets.

GPU-PIM heterogeneous architectures for LLM inference. Prior work partitions LLM operators between GPU and PIM to match compute-to-bandwidth ratios: early designs offload a single operator type [16, 39], while subsequent systems introduce flexible partitioning [15, 29] and extend to MoE Transformers under continuous batching [51]. Orthogonal efforts target long-context inference [25, 26, 49], test-time compute [33], and speculative decoding [31]. Pimba [23] extends GPU-PIM to hybrid models but under static batching. All assume uniform layers or a fixed operator mix with static partitioning. Hy² targets hybrid models whose per-layer operator diversity and batch-size-dependent hardware preference require co-designed microarchitecture, unified state management, and runtime-adaptive scheduling.

9 Conclusion

We presented Hy², a co-design framework for serving hybrid Mamba-Transformer models on GPU-PIM hardware. Our characterization shows that bandwidth-bound SSM scans and attention GEMVs naturally map to PIM, while batch-sensitive linear projections favor GPU—and that this preference shifts at runtime under continuous batching. Hy² exploits this through three co-designed components: workload-guided PIM microarchitecture exploration, a unified memory manager for heterogeneous KV cache and SSM state, and an adaptive per-layer, per-iteration scheduler with SLO enforcement. Across five hybrid models (1.5B–52B parameters, including hybrid-MoE), Hy² achieves $1.51\times$ throughput over Pimba and $2.47\times$ over GPU-only, while reducing P99 TBT to $0.61\times$ of Pimba.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 117–134.
- [2] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small language models are the future of agentic ai. *arXiv preprint arXiv:2506.02153* (2025).
- [3] Aaron Blakeman, Aarti Basant, Abhinav Khattar, Adithya Renduchintala, Akhiad Bercovich, Aleksander Ficek, Alexis Bjorlin, Ali Taghibakhshii, Amala Sanjay Deshmukh, Ameya Sunil Mahabaleshwar, et al. 2025. Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models. *arXiv preprint arXiv:2504.03624* (2025).
- [4] Aaron Blakeman, Aaron Grattafiori, Aarti Basant, Abhibha Gupta, Abhinav Khattar, Adi Renduchintala, Aditya Vavre, Akanksha Shukla, Akhiad Bercovich, Aleksander Ficek, et al. 2025. Nemotron 3 Nano: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. *arXiv preprint arXiv:2512.20848* (2025).
- [5] Aaron Blakeman, Aaron Grattafiori, Aarti Basant, Abhibha Gupta, Abhinav Khattar, Adi Renduchintala, Aditya Vavre, Akanksha Shukla, Akhiad Bercovich, Aleksander Ficek, et al. 2025. NVIDIA Nemotron 3: Efficient and Open Intelligence. *arXiv preprint arXiv:2512.20856* (2025).
- [6] Yuzong Chen, Chao Fang, Xilai Dai, Yuheng Wu, Thierry Tambe, Marian Verhelst, and Mohamed S Abdelfattah. 2025. P3-LLM: An Integrated NPU-PIM Accelerator for LLM Inference Using Hybrid Numerical Formats. *arXiv preprint arXiv:2511.06838* (2025).
- [7] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [8] Tri Dao and Albert Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060* (2024).
- [9] Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, Zijia Chen, Ameya Sunil Mahabaleshwar, Shih-Yang Liu, Matthijs Van Keirsbilck, Min-Hung Chen, Yoshi Suhara, et al. 2024. Hymba: A hybrid-head architecture for small language models. *arXiv preprint arXiv:2411.13676* (2024).
- [10] Cenlin Duan, Jianlei Yang, Rubing Yang, Yikun Wang, Yiou Wang, Lingkun Long, Yingjie Qi, Xiaolin He, Ao Zhou, Xueyan Wang, et al. 2025. HPIM: Heterogeneous Processing-In-Memory-based Accelerator for Large Language Models Inference. *arXiv preprint arXiv:2509.12993* (2025).
- [11] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, Anna Golubeva, Vasudev Shyam, James Whittington, Jonathan Pilault, and Beren Millidge. 2024. The zamba2 suite: Technical report. *arXiv preprint arXiv:2411.15242* (2024).
- [12] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, James Whittington, Jonathan Pilault, Adam Ibrahim, and Beren Millidge. 2024. Zamba: A compact 7b ssm hybrid model. *arXiv preprint arXiv:2405.16712* (2024).
- [13] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).
- [14] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi Iyer, and Reetuparna Das. 2025. Pim is all you need: A cxl-enabled gpu-free system for large language model inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 862–881.
- [15] Yintao He, Haiyu Mao, Christina Giannoula, Mohammad Sadrosadati, Juan Gómez-Luna, Huawei Li, Xiaowei Li, Ying Wang, and Onur Mutlu. 2025. Papi: Exploiting dynamic parallelism in large language model decoding with a processing-in-memory-enabled computing system. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 766–782.
- [16] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 722–737.
- [17] IBM. 2024. Bamba: Inference-Efficient Hybrid Mamba2 Model. IBM HuggingFace. (2024). <https://huggingface.co/blog/bamba>.
- [18] IBM. 2025. IBM Granite 4.0: Hyper-efficient, High Performance Hybrid Models for Enterprise. IBM Blog. (2025). <https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models>.
- [19] Xu Jin, Haotian Zheng, Maohua Nie, Jialin Wang, and C-J Richard Shi. 2025. HCSAs: Hybrid Computing Systolic Arrays for Accelerating Mamba Models with Unified State Space Buffers and Energy-Efficient Dataflow. In *Proceedings of the Great Lakes Symposium on VLSI 2025*. 457–462.
- [20] In-Jun Jung, Gyeongrok Yang, Jaeha Min, and Joo-Young Kim. 2025. HLX: A Unified Pipelined Architecture for Optimized Performance of Hybrid Transformer-Mamba Language Models. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 461–475.
- [21] Hyeoncheol Kim, Taehoon Kim, Taehyeon Park, Donghyeon Kim, Yongseung Yu, Hanjun Kim, and Yongjun Park. 2025. Accelerating LLMs using an efficient GEMM library and target-aware optimizations on real-world PIM devices. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*. 225–240.
- [22] Jiyong Kim, Jaeho Lee, Jiahao Lin, Alish Kanani, Sun Miao, Umit Ogras, and Jaehyun Park. 2025. eMamba: efficient acceleration framework for Mamba models in edge computing. *ACM Transactions on Embedded Computing Systems* 24, 5s (2025), 1–22.
- [23] Wonung Kim, Yubin Lee, Yoonsung Kim, Jinwoo Hwang, Seongryong Oh, Jiyong Jung, Aziz Huseynov, Woong Gyu Park, Chang Hyun Park, Divya Mahajan, et al. 2025. Pimba: A Processing-in-Memory Acceleration for Post-Transformer Large Language Model Serving. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 292–307.
- [24] Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gi-Moon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeonpil Kang, et al. 2022. A 1ynm 1.25 v 8gb 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep learning application. *IEEE Journal of Solid-State Circuits* 58, 1 (2022), 291–302.
- [25] Hyucksung Kwon, Kyungmo Koo, Janghyeon Kim, Woongkyu Lee, Minjae Lee, Gyeonggeun Jung, Hyungdeok Lee, Yousub Jung, Jaehan Park, Yousub Song, et al. 2026. PIMphony: Overcoming Bandwidth and Capacity Inefficiency in PIM-Based Long-Context LLM Inference System. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–21.
- [26] Hyucksung Kwon, Kyungmo Koo, Janghyeon Kim, Woongkyu Lee, Minjae Lee, Hyungdeok Lee, Yousub Jung, Jaehan Park, Yousub Song, Byeongsu Yang, et al. 2024. Lol-pim: Long-context llm decoding with scalable dram-pim system. *arXiv e-prints* (2024), arXiv:2412.
- [27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica.

2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [28] Aakash Lahoti, Kevin Y Li, Berlin Chen, Caitlin Wang, Aviv Bick, J Zico Kolter, Tri Dao, and Albert Gu. 2026. Mamba-3: Improved sequence modeling using state space principles. *arXiv preprint arXiv:2603.15569* (2026).
- [29] Hyojung Lee, Daehyeon Baek, Jimyoung Son, Jieun Choi, Kihyo Moon, and Minsung Jang. 2025. Paise: Pim-accelerated inference scheduling engine for transformer-based llm. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1707–1719.
- [30] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyoungwan Lim, Hyunsung Shin, et al. 2021. Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 43–56.
- [31] Cong Li, Zhe Zhou, Size Zheng, Jiayi Zhang, Yun Liang, and Guangyu Sun. 2024. Specpim: Accelerating speculative inference on pim-enabled system via architecture-dataflow co-exploration. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 950–965.
- [32] Jinhao Li, Shan Huang, Jiaming Xu, Jun Liu, Li Ding, Ningyi Xu, and Guohao Dai. 2024. Marca: Mamba accelerator with reconfigurable architecture. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [33] Sixu Li, Yuzhou Chen, Chaojian Li, Yonggan Fu, Zheng Wang, Zhongzhi Yu, Haoran You, Zhifan Ye, Wei Zhou, Yongan Zhang, et al. 2025. ORCHES: Orchestrated Test-Time-Compute-based LLM Reasoning on Collaborative GPU-PIM Heterogeneous System. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 476–489.
- [34] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirum, Yonatan Belinkov, Shai Shalev-Shwartz, et al. 2024. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887* (2024).
- [35] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2025. Helix: Serving large language models over heterogeneous gpus and network via max-flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 586–602.
- [36] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 932–949.
- [37] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2024. Spotserve: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1112–1127.
- [38] Seungjae Moon, Junseo Cha, Hyunjun Park, and Joo-Young Kim. 2025. Hybe: Gpu-npu hybrid system for efficient llm inference with million-token context window. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 808–820.
- [39] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. Attacc! unleashing the power of pim for batched transformer-based generative model inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 103–119.
- [40] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [41] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading more storage for less computation—a {KVCACHE-centric} architecture for serving {LLM} chatbot. In *23rd USENIX conference on file and storage technologies (FAST 25)*. 155–170.
- [42] Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. 2024. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522* (2024).
- [43] Minseok Seo, Xuan Truong Nguyen, Seok Joong Hwang, Yongkee Kwon, Guhyun Kim, Chanwook Park, Ilkon Kim, Jaehan Park, Jeongbin Kim, Woojae Shin, et al. 2024. Ianus: Integrated accelerator based on npu-pim unified memory system. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 545–560.
- [44] Tencent Hunyuan Team, Ao Liu, Botong Zhou, Can Xu, Chayse Zhou, ChenChen Zhang, Chengcheng Xu, Chenhao Wang, Decheng Wu, Dengpeng Wu, et al. 2025. Hunyuan-turbos: Advancing large language models through mamba-transformer synergy and adaptive chain-of-thought. *arXiv preprint arXiv:2505.15431* (2025).
- [45] vLLM Contributors. 2025. vLLM V1 Hybrid Model Support (Mamba, Jamba, Bamba, Zamba2). GitHub. <https://github.com/vllm-project/vllm>.
- [46] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, et al. 2024. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887* (2024).
- [47] Yimin Wang, Yue Jiet Chong, and Xuanyao Fong. 2025. LEAP: LLM inference on scalable PIM-NoC architecture with balanced dataflow and fine-grained parallelism. In *2025 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [48] Yuting Wu, Ziyu Wang, and Wei D Lu. 2024. Pim gpt a hybrid process in memory accelerator for autoregressive transformers. *npj Unconventional Computing* 1, 1 (2024), 4.
- [49] Liang Yan, Xiaoyang Lu, Xiaoming Chen, Yinhe Han, and Xian-He Sun. 2025. Pyramid: Accelerating llm inference with cross-level processing-in-memory. *IEEE Computer Architecture Letters* (2025).
- [50] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)*. 521–538.
- [51] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1429–1443.
- [52] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [53] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model

1379	serving. In <i>18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)</i> . 193–210.	1432
1380		1433
1381	[54] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A memory-based acceleration via software-hardware	1434
1382	co-design for transformer. In <i>2022 IEEE International Symposium on</i>	1435
1383	<i>High-Performance Computer Architecture (HPCA)</i> . IEEE, 1071–1085.	1436
1384	[55] Jingwei Zuo, Maksim Velikanov, Dhia Eddine Rhaïem, Ilyas Chahed,	1437
1385	Younes Belkada, Guillaume Kunsch, and Hakim Hacid. 2024. Falcon	1438
1386	mamba: The first competitive attention-free 7b language model. <i>arXiv</i>	1439
1387	<i>preprint arXiv:2410.05355</i> (2024).	1440
1388		1441
1389		1442
1390		1443
1391		1444
1392		1445
1393		1446
1394		1447
1395		1448
1396		1449
1397		1450
1398		1451
1399		1452
1400		1453
1401		1454
1402		1455
1403		1456
1404		1457
1405		1458
1406		1459
1407		1460
1408		1461
1409		1462
1410		1463
1411		1464
1412		1465
1413		1466
1414		1467
1415		1468
1416		1469
1417		1470
1418		1471
1419		1472
1420		1473
1421		1474
1422		1475
1423		1476
1424		1477
1425		1478
1426		1479
1427		1480
1428		1481
1429		1482
1430		1483
1431		1484